

TP DevOps : Déploiement d'une Architecture Mobile

Motivation : Pourquoi ce TP ?

Avant de commencer, observons deux situations classiques que la méthodologie DevOps permet d'éviter.

Scénario A : Le fameux "Ça marche chez moi"

Le Contexte : Ahmed (Backend) et Selma (Mobile) collaborent. Ahmed a fini l'API sur son PC. Il envoie le code à Selma. **Le Problème :** Selma obtient une erreur : `Connection refused`. Elle n'a pas la même base de données qu'Ahmed, ni la même version de Java. **Résultat :** Ils perdent 4 heures à configurer le PC de Selma.

Scénario B : Le Chatbot de Master

Le Contexte : Sofiane (Étudiant M2) a développé un Chatbot Python/TensorFlow. **Le Problème :** Lors d'une démo, rien ne marche sur les PC du labo car les bibliothèques manquent ou l'OS est différent. **La solution :** Avec **Docker**, le Chatbot se serait lancé à l'identique partout.

Contexte et Objectifs

L'objectif est de mettre en place une chaîne DevOps pour une architecture client-serveur.

Modalités de rendu

Rapport individuel à envoyer à a_ouared@esi.dz avec l'objet : [TP DevOps] TP2 Nom Prénom. *Ce TP sera pris en compte juste après les examens du premier semestre (Janvier 2026).*

Note Importante : Choix du Backend

Ce TP nécessite une API Backend pour fonctionner. Vous avez deux options :

1. **Utiliser votre propre projet (Recommandé)** : Si vous avez déjà développé un Backend (Node.js, Python, PHP, Java...) pour un autre module, utilisez-le ! Adaptez simplement les commandes Docker à votre technologie.
2. **Utiliser le projet de référence** : Si vous n'avez pas de backend disponible, vous pouvez utiliser celui fourni spécifiquement pour ce TP (disponible sur GitHub, lien dans le Module 2).

1 Prérequis et Environnement Technique

Si ce n'est pas déjà fait, installez le moteur Docker sur votre machine. **Remplissez le tableau ci-dessous** en indiquant la version installée sur votre poste.

Outil	Version	Rôle et Interaction dans le projet
Docker		
Docker Compose		
SGBD (Votre choix)		
Langage Backend		
Android SDK		
Git		

2 Module 1 : Persistance des données

Si vous utilisez MySQL ou un autre SGBD, adaptez l'image Docker et les commandes SQL ci-dessous.

Lancez un conteneur de base de données (PostgreSQL ou votre choix) en vous référant à la documentation officielle.

1. Connectez-vous au conteneur et initialisez la base de données pour l'application mobile. Exemple pour PostgreSQL :

```
CREATE EXTENSION "uuid-ossf";
CREATE TABLE mobile_sessions (
    device_id UUID NOT NULL PRIMARY KEY,
    model_name varchar(100) NOT NULL,
    last_login TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. Effectuez un redémarrage du conteneur. Vérifiez si la table `mobile_sessions` est toujours présente. Où sont physiquement stockées les données actuelles ?
3. Supprimez totalement le conteneur puis recréez-le. Que constatez-vous concernant les données ?
4. Relancez un conteneur SGBD en appliquant strictement les contraintes de production suivantes :
 - Mappage de port : La base doit être joignable sur un port spécifique (ex : 2345 ou 3306) ;
 - Nom de la base : `mobile_backend_db` ;
 - Authentification : Utilisateur `admin` et mot de passe `securePass123` ;
 - Contraintes ressources : Limite de RAM à 50 MB et max 1 CPU ;
 - Persistance : Les données doivent être sauvegardées dans un dossier local nommé `db_data` à la racine.
5. Analysez la consommation de ressources du conteneur (commande `stats`). Expliquez brièvement les métriques affichées (I/O, Mem, CPU).

3 Module 2 : Orchestration (Docker Compose - Partie 1)

6. Récupérez le code source de l'API Backend.
 - **Option A (Perso)** : Utilisez votre propre code source.
 - **Option B (Référence)** : Clonez le dépôt suivant :
<https://github.com/ouared14/Mobile-Programming-2025/tree/main/TP-RestFullAPI>
7. Lancez l'API Backend localement (sans Docker). Testez son bon fonctionnement (ex : `http://localhost:8080/health` ou votre endpoint). Pourquoi, sans la base de données lancée, le statut de l'application est-il en erreur ?
8. Installez Docker Compose sur votre machine si nécessaire.
9. Rédigez un fichier `docker-compose.yml`. Il doit définir le service de base de données configuré pour être compatible avec votre API Backend. Lancez le service, puis relancez votre API locale. Vérifiez que tout fonctionne.

4 Module 3 : Conteneurisation de l'API (Dockerfile)

10. Construisez l'artefact de votre API (JAR, exécutable, script...). Exécutez-le manuellement et assurez-vous qu'il se connecte à la base.
11. Créez un répertoire `docker` à la racine. Écrivez un Dockerfile optimisé pour votre API Backend. Important : Les paramètres de connexion à la base (URL, User, Password) doivent être injectables via des variables d'environnement.
12. Construisez l'image Docker de votre API et lancez un conteneur à partir de celle-ci.
13. Automatisation du build : Modifiez votre script de build (Gradle, npm scripts, Makefile...) pour permettre de construire directement l'image Docker depuis l'outil de build.

5 Module 4 : Orchestration complète (Docker Compose - Partie 2)

15. Enrichissez votre fichier `docker-compose.yml`. Ajoutez-y le service correspondant à votre API Backend (en plus de la base de données). Réseau : Assurez-vous que l'API et la base de données communiquent via un réseau Docker privé et isolé. *Note pour le test Android* : Si vous utilisez un émulateur Android, rappelez-vous que `localhost` désigne l'émulateur lui-même. L'API sera accessible via l'IP de votre machine hôte (souvent 10.0.2.2).

6 Module 5 : Registre d'images (Docker Hub)

1. Créez un compte sur Docker Hub si vous n'en possédez pas.
2. Taguez l'image de votre API Backend selon la convention :
`<votre-login>/mobile-backend-api`.
3. Connectez-vous en ligne de commande (`login`) et poussez (`push`) l'image sur le registre public.

7 Module 6 : Intégration Continue (GitHub Actions)

Cette étape est facultative mais fortement recommandée.

20. Mettez en place un pipeline CI/CD via GitHub Actions (fichier `.github/workflows/main.yml`) qui réalise les actions suivantes à chaque push :
 - Build de l'API Backend et push de l'image Docker sur le Hub.
 - Build de l'application Android (génération de l'APK).

8 Module 7 : Infrastructure Déploiement (Ops)

21. Provisionnez 4 Machines Virtuelles (VMs) Linux. Nommez-les : `lb-01` (Load Balancer), `api-01`, `api-02` (Serveurs API) et `db-01` (Base de données).
22. Sur `db-01` : Installez votre SGBD (Postgres, MySQL...).

23. Sur **api-01** et **api-02** : Déployez votre API Backend. Configurez-la pour se lancer automatiquement (service **systemd**). Vérifiez que le point de terminaison de santé répond "UP" sur les deux serveurs.
24. Sur **lb-01** : Installez HAProxy (ou Nginx). Configurez-le pour répartir la charge (Load Balancing) des requêtes venant de l'application Android vers **api-01** et **api-02**.
25. Test final : Configurez votre application Android (sur votre téléphone ou émulateur) pour pointer vers l'adresse IP de **lb-01**. Vérifiez que l'application fonctionne correctement.

9 Pour aller plus loin (Bonus)

26. Automatisez le provisionnement et la configuration des VMs (Module 7) en utilisant Ansible ou Puppet.